Java JUMP

summerbootcamp

Final Project and Practice

Agenda

1. **Final Project: Combination of all elements**
   (data model, service layer, API, repositories, application testing)
2. **Project Presentation**

Final project

💡Final Project: "MyEvents" - Local Event Management
An application that allows you to create events (e.g. concerts, workshops, meetings), assign participants, manage locations and register registrations.

📆Work plan (10h division):

Time	Tasks
1h	Entity Design + JPA Relationships
1h	Repository configuration + test data
2h	API for events and participants + ResponseEntity, validation
1h	Handling of records (registration logic + validations)
1h	Exception Handling (@ExceptionHandler)
1h	Swagger + REST documentation
2h	Thymeleaf Views: Event List, Sign Up, Forms
1h	Unit and integration testing

✅Project Plan: MyEvents – Event Management App

🗂️1. Preparing the environment (0.5h)

- Create a new Spring Boot project with dependencies:
  - Spring Web
  - Spring Data JPA
  - MySQL Driver
  - Thymeleaf
  - Spring Boot DevTools
  - Spring Validation
  - Spring Boot Test
  - OpenAPI / Swagger
- Configure MySQL connection (application.properties)
- Create the project structure: model, repository, service, controller, dto, view

---

🧱2. JPA Data Model and Relationships (1h)

Entities:

- Event: id, name, description, date, capacity, Location, Set<Registration>
- Location: id, name, city, address
- Participant: id, name, email, Set<Registration>
- Registration: id, event, participant, registrationDate

Relationships:

- Event→Location (@ManyToOne)
- Event↔Participant by Registration (@OneToMany)
- Registration→Event, Participant (@ManyToOne)
- Add @JoinColumn, CascadeType, mappedBy where needed

💾3. Database and repositories (0.5h)

- Create repositories:
  - EventRepository
  - ParticipantRepository
  - LocationRepository
  - RegistrationRepository
- Generate test data (e.g. in CommandLineRunner)
- Check if the database is populated after launching

🧠4. Service layer (1.5h)

- EventService: CRUD, event list, seat limit check
- ParticipantService: participant registration
- RegistrationService: registering a participant for an event (with validation: e.g. if there is a place, if not registered twice)

🌐5. REST API (1.5h)

Controllers:

- EventController (REST):GET /api/events, GET /api/events/{id}, POST, PUT, DELETE
- ParticipantController:POST /api/participants, GET /api/participants/{id}
- RegistrationController:POST /api/events/{eventId}/register/{participantId}
- Use ResponseEntity<?> everywhere

❗6. Error handling (0.5h)

- Create exceptions: EventNotFoundException, RegistrationFullException, AlreadyRegisteredException

- Handle errors globally (@ControllerAdvice)
- Return readable JSON responses with HTTP codes

🔐7. Data validation (0.5h)

- @NotBlank and @Email in Participant
- @NotBlank, @Future, @Min(1) in Event
- Add @Valid in controllers + checks for invalid data

📜8. Swagger / OpenAPI (0.5h)

- Add springdoc-openapi-ui
- Check under /swagger-ui.html
- Add descriptions to endpoints: @Operation, @ApiResponse

🖥️9. Frontend with Thymeleaf (1.5h)

Views:

- /events – list of events
- /events/new – event addition form
- /events/{id} – event details + "Sign up" button
- /participants/new – participant registration
- /events/{id}/registrations – list of registered

View controllers:

- EventViewController, RegistrationViewController – MVC versions

---

🧪10. Unit and integration testing (1h)

- REST endpoint tests (@WebMvcTest + MockMvc)
- Service Tests (@SpringBootTest, @DataJpaTest)
- Validation and exception testing

🗒️11. Project presentation (last 0.5h)

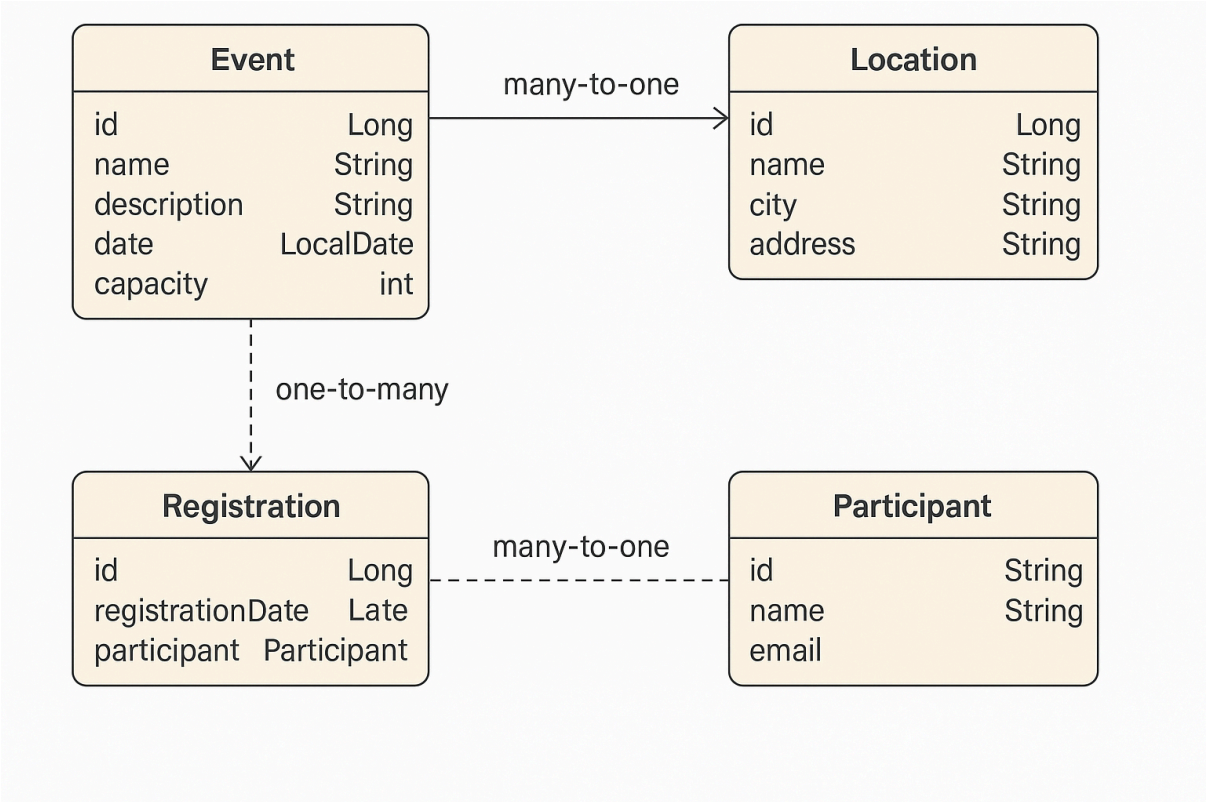
- Show API action in Swagger
- Show views Thymeleaf
- Go through the code: structure, relationships, validations
- Tell us about your problems/interesting facts

📦Bonus/Extensions (if time remains):

- Pagination and event filtering
- Export participants to CSV

- Sending an e-mail confirming registration (e.g. JavaMailSender)
- Integration with Google Maps by address

Example solution: myEvent.zip


Event
id Long
name String
description String
date LocalDate
capacity int
many-to-one
Location
id Long
name String
city String
address String
one-to-many
Registration
id Long
registrationDate Late
participant Participant
many-to-one
Participant
id String
name String
email